

SIPLACE Setup Report Generator

Application Technical Documentation

Structured technical reference covering architecture, XML processing, SQL storage, reports, deployment, testing and maintenance.

Document information	Value
Application	SIPLACE Setup Report Generator
Type	Windows Forms desktop application
Language	C#
Target	.NET 8 / Windows
Database	SQL Server Express
Libraries	EF Core, LINQ to XML, ClosedXML, QuestPDF
Version	1.0
Date	23 September 2026

Document Contents

1. Document Purpose
2. Product Overview
3. Requirements and Scope
4. Functional Workflow
5. Solution Architecture
6. Application Components
7. Data Model and Database Design
8. XML Structure and Mapping
9. XML Import and Parsing
10. Recipe and Setup Data Processing
11. Report Generation
12. User Interface
13. Multiple XML Import
14. Error Handling and Validation
15. Configuration and Environment
16. Installation and Setup
17. Executable Build and Deployment
18. Testing and Verification
19. Troubleshooting
20. Security and Data Considerations
21. Performance and Scalability
22. Maintenance and Extension
23. Requirement Traceability
24. Current Limitations
25. Technical Summary

1. Document Purpose

This document describes the SIPLACE Setup Report Generator as an application rather than a project report. It explains purpose, architecture, data flow, configuration, deployment, testing and maintenance.

It is intended for developers, testers, reviewers, users and technical interviewers.

2. Product Overview

The application converts SIPLACE XML recipe information into structured recipe/setup records, stores them in SQL Server and generates setup reports in Excel and PDF.

Capabilities include XML import, parsing, SQL persistence, recipe listing, grouping, Excel export, PDF export and self-contained Windows publishing.

3. Requirements and Scope

The assignment requires a C# Windows application that imports XML, stores data in SQL Server, lists imported recipes and regenerates Excel/PDF reports. It also specifies the required header fields, detail fields and grouping key.

4. Functional Workflow

XML files → Import → XMLParser → Recipe + SetupDetail objects → Entity Framework Core → SQL Server → Recipe selection → ReportGenerator → grouping/aggregation → Excel or PDF.

XML → Parser → Objects → EF Core → SQL Server → Selection → Grouping → Excel/PDF

5. Solution Architecture

The solution separates presentation, models, data access, parsing, report logic and output generation. This prevents XML parsing from depending on report formatting and keeps exporters independent from source XML.

6. Application Components

Recipe and SetupDetail represent data. SiplaceContext manages EF Core persistence. XMLParser reads XML. ReportGenerator queries and groups data. ExcelExporter and PdfExporter create output. MainForm coordinates the workflow.

7. Data Model and Database Design

Recipes is the parent entity and SetupDetails is the child entity. RecipeId is the primary key of Recipes and the foreign key in SetupDetails. The relationship is one-to-many with cascade delete.

8. XML Structure and Mapping

Recipe@Name maps to RecipeName; Line@Name to LineName; Board@Name to ModelName; Station@Name to MachineName; Position provides component/reference data; Pickup provides Table, Track and FeederType.

9. XML Import and Parsing

XMLParser loads the document using XDocument, finds Line and Board descendants, collects Position and Pickup elements, and obtains MachineName from the Station ancestor. Parsing failures are surfaced as import errors.

10. Recipe and Setup Data Processing

SetupDetails are queried for the selected RecipeId and grouped by MachineName, Table, Track, PartNumber and FeederType. Quantity is summed and reference designators are joined.

```
GroupBy(MachineName, Table, Track, PartNumber, FeederType)
Quantity = Sum(Quantity)
ReferenceDesignators = comma-separated values
```

11. Report Generation

ClosedXML creates the Excel workbook and QuestPDF creates the PDF. Both contain the required recipe header and setup detail fields. The PDF layout is designed for A4 landscape.

12. User Interface

The main UI provides Import, recipe selection, and Generate Excel/PDF actions. The recipe grid displays database records and the selected recipe becomes the report source.

13. Multiple XML Import

Multiple files can be selected in one import operation. Each file is parsed and stored as its own recipe with associated setup details, after which the recipe list is refreshed.

14. Error Handling and Validation

Main scenarios include malformed XML, missing attributes, database connection problems, missing tables, no selected recipe and export failures. Database/schema alignment was also verified during implementation.

15. Configuration and Environment

The application uses SQL Server Express through Entity Framework Core. The current EF context contains the environment-specific connection configuration.

16. Installation and Setup

Install SQL Server Express, create/verify the SIPLACE database with the SQL script, verify Recipes and SetupDetails, restore packages, build and run, then test with the sample XML.

17. Executable Build and Deployment

The application was published as a self-contained Windows x64 Release build using dotnet publish -c Release -r win-x64 --self-contained true. The complete publish directory should be distributed.

18. Testing and Verification

Testing covered XML import, recipe metadata, SQL storage, recipe selection, grouping, Excel generation, PDF generation, multiple XML imports and the published executable.

Test	Status
Import valid XML	Passed
Extract Recipe/Line/Model	Passed
SQL storage	Passed
Grouping	Passed
Excel	Passed
PDF	Passed
Multiple XML import	Passed
Published EXE	Passed

19. Troubleshooting

Common issues include SQL instance/database mismatch, wrong schema, EF key mismatch, blank ModelName in old records, missing setup rows and incorrect report output paths.

20. Security and Data Considerations

The current solution is a local Windows desktop application. Production deployment should protect connection details, use appropriate SQL permissions and follow organizational data-handling requirements.

21. Performance and Scalability

For larger workloads, future improvements include XmlReader streaming, asynchronous operations, batch inserts, indexing, background processing and incremental report generation.

22. Maintenance and Extension

New XML fields require model/parser/database/exporter changes. Grouping changes belong in ReportGenerator. New output formats should use separate exporter services.

23. Requirement Traceability

Import maps to MainForm/XMLParser; persistence maps to SiplaceContext/SQL; grouping maps to ReportGenerator; Excel/PDF map to exporters; executable and documentation map to the final deliverables.

Requirement	Implementation
Import XML	MainForm + XMLParser

Requirement	Implementation
SQL storage	SiplaceContext + SQL
Recipe list	MainForm grid
Required report fields	Models + exporters
Grouping	ReportGenerator
Excel/PDF	Exporters
Executable	Release publish
Documentation	This document

24. Current Limitations

The parser assumes the expected SIPLACE XML structure and processes Position/Pickup information in sequence. The application is local rather than a multi-user web service. Authentication, automated test projects and production configuration externalization are outside the current scope.

25. Technical Summary

The implemented path is XML → LINQ to XML → Recipe/SetupDetail → EF Core → SQL Server → recipe selection → LINQ grouping/aggregation → Excel/PDF.